

Premier rendu de Go

Crée par Google en 2009 :)

Go	Python
- Compilé	- Interprété
- C++ (syntaxe semblable?)	- Syntaxe facile
- C# mais sans le framework	- Moins de performance

Les langages interprétés ont une **couche intermédiaire** avec le CPU mais sont **moins performant** que ceux dit compilé comme le Go, le C...

Compilé = utilisé lorsqu'on est plus **proche de la machine** (électronique , jeux vidéo...)

Les langages interprétés ne sont pas les plus complexes ou rigide, mais sont obligé de recompilé et peuvent entraîner des fuites de mémoire.

Disruptif : qui crée une cassure (iphone avec ces avancées rassemblés en 1)

Objectif d'un langage pour être disruptif : bas niveau, web, mobile => performances

L'ia ne saurait pas le faire actuellement, elle se base sur l'existant

Langage utilisé pour l'ia : Python

Langage objet : Classe et héritage, réutilisabilité mais a le défaut d'être trop lourd.

On le retrouve la notion de **composition**, d'un langage interprété, dans le Go qui est compilé.

Avantage du Go :

- **Simplicité** : plus simple à lire/écrire (lire commentaire/doc en premier)
- **Rigidité** pour simplification d'utilisation, lecture... (import fmt)
- **Rapidité** : exécution, compilation... (si les contraintes du go sont respecté ont gagne en rapidité)

Concurrence :

2 threads (processus) qui accède à la même mémoire	> Mémoire ! (RAM) Données compromises Données bloqués Deadlock	> Plusieurs taches Multi threading
--	---	---------------------------------------

Threads partageant la mémoire

- risques
- Go préfère des tâches séparées qui communiquent

“Do not communicate by sharing memory; share memory by communicating.” → Go essaie de simplifier la concurrence en évitant les accès concurrents à la même mémoire.

Le pragmatisme : 1 seul façon de faire

- **stabilité**
- **comptabilité**

Donc c'est le langage qui a été développé pour être le plus efficace possible, avoir un temps de compilation rapide, être super rapide sur tout les points...

Performance nettement mieux que python et js

Mots clé syntaxique :

- Java : 55
- Python : 35
- Go : 25

5 millions d'utilisateur de Go

Docker, Kubernetes, Terraform, Prometheus écrit en Go

Utiliser Go pour :

- microservices et Api Rest
- outils CLI, commandes lines
- agent DevOps
- iaas

Jeu Go ou Rust : Go pour les performances

Pour android : kotlin

Pratique:

Hello world

```
test.go
C:\wamp64> www > M2IW > Go > test.go > ...
1 package main
2
3 import "fmt"
4
5 func main() {
6     // message et commentaires
7     fmt.Println("Langage installé : Hello World!")
8 }
```

```
Hugo7@DESKTOP-7MLA92A MINGW64 /c/wamp64/www/M2IW/Go
$ go run test.go
Langage installé : Hello World!
```

64²

```
test.go
C:\wamp64> www > M2IW > Go > test.go > ...
1 package main
2 import "fmt"
3
4 func main() {
5     // message et commentaires
6     fmt.Println(64 * 64)
7 }
8
```

```
Hugo7@DESKTOP-7MLA92A MINGW64 /c/wamp64/www/M2IW/Go
$ ./test.exe
4096
```

Date de naissance :

```
C:\wamp64> www > M2IW > Go > test.go > main
3 import (
4     "fmt"
5     "time"
6 )
7
8 func main() {
9     // message et commentaires
10    naissance := time.Date(2002, time.March, 11, 0, 0, 0, 0, time.UTC)
11    maintenant := time.Now()
12
13    years := maintenant.Sub(naissance).Hours() / 24 / 365
14    mois := int(years*12) % 12
15    jours := int(years*365)%365 - int(mois*30)
16    heures := int(years*365*24) % 24
17
18    fmt.Printf("Je suis né le 11 mars 2002.\n")
19    fmt.Printf("J'ai %d ans, %d mois, %d jours et %d heures\n", int(years), mois, jours, heures)
20 }
21
```

```
Hugo7@DESKTOP-7MLA92A MINGW64 /c/wamp64/www/M2IW/Go
$ go run test.go
Je suis né le 11 mars 2002.
J'ai 24 ans, 2 mois, 22 jours et 15 heures
```

go run : la commande va compiler le fichier en binaire temporaire, l'exécuter et supprimer le fichier temporaire

Correspond plus ou moins à l'équivalent de « go build X.go -o /tmp/random-tmp-folder/exe && /tmp/random-tmp-folder/exe »

go build : contrairement à go run, il va seulement compiler le fichier en binaire temporaire et créer l'exécutable. Quelques options :

- « -o » : choisit le nom du fichier de sortie.
- « -v » : affiche les packages compilés.
- « -x » : affiche les commandes internes exécutées
- « . » : compiler le projet entier

go build -ldflags -s -w :

- « -ldflags » : passe des options au linker
- « -s » : supprime la table des symboles
- « -w » : supprime les informations de debug

A pour le but final de réduire la taille du fichier binaire puisqu'on enlève certains caractères. Et sera donc plus petit qu'un build normal mais moins pratique à debugger

Les types en Go :

Entiers : nombres sans virgules

int 8 (de -128 à 127)		uint 8 (0 à 255)
int 16		uint 16
int 32		uint 32
...		...

Mais on préfère utiliser les uint

Flottants : nombres avec virgules

float 32

float 64 : meilleure précision, souvent utilisé par défaut

Booléens :

seulement true ou false

déclaration : var bool1 = true

String : contient du texte

variables non déclarée = string

déclaration : var texte = « mon texte »

On peut les additionner, les comparer...

Les strings sont également indexables, on peut les rechercher (optimisation)

Spéciaux :

byte : alias de type uint 8

rune : alias de type int 32

Déclarer constante avec const

Déclarer variable avec « var »

Déclarer et mettre la valeur : «:= »

Iota :

Les iota sont utilisés en lien avec un bloc const, dans le but de simplifier l'incrémentation des constantes à l'intérieur de ce bloc.

Il commence à 0 et s'incrémente à chaque nouvelle déclaration d'une constante.

Les lignes vides ou les commentaires n'incrémentent pas.

Dans le schéma :

- sunday : iota 0

- _ : iota 1

- Monday : iota 2

...

On les utilise pour avoir des constantes propres et facile à maintenir, créer des enum et dans le schéma ça évite de les marquer à la main (comme je l'ai fais au dessus).

Le schéma qu'on retrouve sur le wiki : <https://go.dev/wiki/Iota>

```
type Weekday int

const (
    Sunday Weekday = iota + 1  ➔ iota: 0 ~ Sunday : 1
    _           ➔ iota: 1 ~ iota increased
    // comments ➔ iota: 1 ~ skip: comment
                ➔ iota: 1 ~ skip: empty line

    Monday      ➔ iota: 2 ~ Monday : 3
    Tuesday     ➔ iota: 3 ~ Tuesday : 4
    Wednesday   ➔ iota: 4 ~ Wednesday: 5
    Thursday    ➔ iota: 5 ~ Thursday : 6
    Friday       ➔ iota: 6 ~ Friday : 7
    Saturday    ➔ iota: 7 ~ Saturday : 8
)
```

2ème cours

3 façons de déclarer une variables

- « var xxx string = valeur » : force le type
- « var xxx = valeur » : déduit le type
- « xxx:= valeur » : que dans une fonctions

Plusieurs variables à la fois :

« x, y := valeur1, valeur2 »

Variables avec plusieurs déclarations groupées :

```
var {
    xxx string = 'valeur1'
    yyy int = 11
}
```

Exo 1 :

 **EXERCICE NOTÉ 1 — Calculateur d'IMC et de catégorie** [15 min]

Contexte : L'IMC (Indice de Masse Corporelle) est utilisé en médecine. C'est un premier exercice pratique mêlant types, variables et constantes.

1. Déclarez deux variables float64 : poids (ex. 70.5 kg) et taille (ex. 1.75 m)
2. Créez 4 constantes : IMCMaigre=18.5, IMCNormal=25.0, IMCSurpoids=30.0
3. Calculez l'IMC = poids / (taille * taille)
4. Affichez l'IMC avec 2 décimales (fmt.Printf avec %.2f)
5. Affichez la catégorie : Maigre / Normal / Surpoids / Obésité
6. BONUS : déclarez une constante Nom avec votre prénom et personnalisez l'affichage

En Go par de surcharge de fonction

Retour multiple : le pattern idiomatique (res, erreur)

Vérifier l'erreur :

```
résultat, err := mafonction(x, y)
if err != ...
```

Ignorer valeur de retour avec l'underscore _ :

```
res2, _ := mafonction(x, y)
```

Fonction closure : fonction qui peut accéder aux variables de la portée où elle a été créée, même après la fin de cette portée. Les closures s'appuient sur les valeurs de fonctions et les fonctions anonymes ; elles permettent de conserver un état entre les appels, d'éviter les variables globales et d'écrire du code plus flexible.

→ ce souvent d'où elle a été créée

middleware http, callback

for → seule boucle en go

```
for i := 0 ; i < 5 ; i++ { }
```

```
for i < 200 { }
```

```
for { }
```

Exo 2 calculatrice :



EXERCICE NOTÉ 2 — Calculatrice avec closures et validation [20 min]

Contexte : Une calculatrice CLI qui démontre les fonctions, retours multiples et closures.

1. Créez une fonction `operer(a, b float64, op string) (float64, error)` qui supporte '+', '-', '*', '/' — retourne une erreur pour / par 0 ou op inconnue
2. Créez une fonction `creerOperation(op string) func(float64, float64) float64` qui retourne une closure pour chaque opération
3. Dans `main()`, faites une boucle 'while true' qui :
 - Lit deux nombres et une opération depuis l'entrée standard (`fmt.Scan`)
 - Affiche le résultat ou l'erreur
 - Sort si l'opération est 'quit'
4. Testez avec : `10 5 + → 15 | 8 0 / → erreur | 3 4 * → 12`

Rendu : fichier `calculatrice.go` sur GitHub

Indices : `switch op { case '+': ... } | fmt.Scan(&a, &b, &op) | for { if op == 'quit' { break } }`

Closure != récursif

Closure garde la valeur

Switch case → fallthrough : ne s'arrête pas au premier cas trouvé, continue à regarder les autres conditions. Permet d'éviter les erreurs les plus communes des switch case

Exo libre

Structure : classe
on peut mettre un slice dedans aussi

Instancier structure :

- positionnelle : dans une fonction avec notre :=
- nommée : pareil mais champs de style nom : « valeur »

Intérêt pointeur/receveur : permet à une méthode de modifier directement la structure d'origine en recevant son adresse (*Type) au lieu d'une copie. Evite de copier de grosses structures en mémoire.

Les tags sont des annotations ajoutées aux champs d'une structure (struct) pour contrôler leur comportement, notamment avec le JSON.

Ils permettent par exemple de renommer un champ (json:"prenom"), d'ignorer les champs vides avec omitempty, ou de ne jamais sérialiser certaines données sensibles comme un mot de passe (json:"-").

Exo 3 :

 **EXERCICE NOTÉ 3 — Système de contacts : Personne, Employé, Étudiant [20 min]**

Contexte : Modéliser un annuaire d'entreprise avec différents types de contacts.

1. Créez une struct `Personne` { `Prenom` string; `Nom` string; `Age` int; `Email` string } avec la méthode `NomComplet()` string et `Presentation()` string
2. Créez `Adresse` { `Rue`, `Ville`, `CodePostal` string } avec `Format()` string
3. Créez `Employe` en embeddant `Personne` et `Adresse` + champs `Poste` string, `Salaire` float64
Ajoutez la méthode `FicheEmploye()` string qui affiche toutes les infos
Ajoutez `AugmenterSalaire(pct float64)` qui modifie le salaire
4. Créez `Etudiant` en embeddant `Personne` + champs `Promo` string, `Moyenne` float64
Ajoutez la méthode `MentionObtenue()` string (TB/B/AB/P selon moyenne)
5. Dans `main()`, créez 2 employés et 2 étudiants, affichez leurs fiches

Rendu : `contacts.go` sur GitHub

Indices : (e *`Employe`) pour modifier le salaire | `switch { case moy >= 16: ... } | fmt.Sprintf` pour construire des chaînes

Pas de public, private, protected en Go

Majuscule = public / Minuscule = privé

minimaliste pour garder le langage simple et lisible

 **EXERCICE NOTÉ TP1 — TechShop — Boutique CLI [45 min autonome + rendu GitHub]**

Contexte : Vous êtes développeur chez TechShop, une boutique en ligne de matériel informatique. Vous devez créer un gestionnaire de catalogue en ligne de commande.

=== DONNÉES ===

- Struct `Produit` : `ID` int, `Nom` string, `Marque` string, `Prix` float64, `Stock` int, `Categorie` string (actif bool)
- Struct `Catalogue` avec une slice de `Produits`

=== FONCTIONNALITÉS (méthodes sur `Catalogue`) ===

- `AjouterProduit(p Produit) error` — erreur si ID dupliqué
- `TrouverParID(id int) (Produit, error)`
- `TrouverParCategorie(cat string) []Produit`
- `AppliquerReduction(categorie string, pct float64) int` — retourne nb de produits modifiés
- `Vendre(id int, qte int) error` — réduit le stock, erreur si insuffisant
- `Rapport() string` — résumé : nb produits, valeur totale du stock

=== MAIN ===

- Menu CLI interactif : [1] Ajouter [2] Chercher [3] Soldes [4] Vendre [5] Rapport [0] Quitter
- Pré-remplissez 5 produits high-tech réalistes (iPhone, MacBook, etc.)
- Gérez toutes les erreurs proprement

=== RENDU ===

- Fichier : `tp1_techshop.go` (tout dans un seul fichier)
- Commande : `git init + git add + git commit + git push` sur votre GitHub
- URL du repo à envoyer dans le chat Teams